

Mapa de Sabores - Propuesta de Proyecto y Especificación Técnica

Proyecto Final Desarrollo de Sistemas Web

Alumno: Diego Rafael Guaraz

1. Resumen

Mapa de Sabores con IA es un sistema web interactivo de descubrimiento gastronómico basado en una arquitectura híbrida de **Base de Datos Relacional + Inteligencia Artificial (IA)**.

El proyecto resuelve el problema del maridaje e innovación culinaria mediante una representación en forma de **Grafo de Sabores** interactivo y dinámico. Permite a profesionales de la cocina y aficionados explorar combinaciones de ingredientes basados en afinidad química y culinaria, consultar recetas reales integradas y recibir justificaciones organolépticas generadas por modelos de lenguaje (LLM).

Principales pilares de ingeniería:

1. **Certeza en el Core (Base Relacional Estable):** La red de sabores reside en una base de datos PostgreSQL optimizada con índices compuestos bidireccionales. La estructura del grafo se define por datos curados y validados, no por generación en tiempo real de un modelo de lenguaje.
 2. **Pipeline de Datos Sintéticos Offline:** Un pipeline de ingeniería de prompts sobre LLMs compila y cura un dataset inicial de 200–300 ingredientes y miles de pares de afinidad.
 3. **Capa de IA Desacoplada e Intercambiable:** Un servicio backend agnóstico en FastAPI permite alternar entre proveedores cloud (Google Gemini, OpenAI) y ejecución 100% local (Ollama / Llama 3.2 3B).
 4. **Visualización React en 2D:** Renderizado dinámico de nodos y aristas mediante `react-force-graph-2d` con experiencia de usuario fluida y paneles laterales descriptivos.
-

2. Marco Académico y Planteamiento del Problema

2.1 Problema Identificado

El descubrimiento de combinaciones de ingredientes (maridaje o *flavor pairing*) tradicionalmente ha dependido de la intuición empírica o de enciclopedias culinarias estáticas. Aunque existen teorías científicas de maridaje de sabores

(compartir compuestos aromáticos clave), no existen herramientas web abiertas e interactivas en español que combinen:

- Exploración visual intuitiva en forma de grafo dinámico.
- Explicaciones organolépticas personalizadas en lenguaje natural.
- Búsqueda en tiempo real de recetas aplicadas.

2.2 Objetivos del Proyecto

- **Objetivo General:** Desarrollar una aplicación web full-stack funcional y escalable que permita explorar redes de sabores e interacciones de ingredientes asistida por Inteligencia Artificial.
- **Objetivos Específicos:**
 1. Diseñar un esquema relacional optimizado en PostgreSQL para modelar grafos bidireccionales de afinidad, con consultas de vecinos resueltas mediante índices compuestos.
 2. Implementar un pipeline offline en Python para la generación, validación y sanitización de un dataset sintético de afinidades culinarias.
 3. Crear una API REST en FastAPI con arquitectura limpia y abstracción del proveedor de LLM.
 4. Desarrollar una interfaz de usuario interactiva en React utilizando la librería `react-force-graph-2d`.
 5. Integrar autenticación JWT para áreas personalizadas de usuarios (guardar combinaciones favoritas y recetas).

2.3 Defensa

Las decisiones de diseño se fundamentan en criterios estratégicos de ingeniería de software:

- **PostgreSQL vs. Neo4j:** Se optó por PostgreSQL debido a que en una red de 300 a 1,000 ingredientes las consultas de 1 o 2 saltos (*hops*) no justifican la sobrecarga operativa y de consumo de memoria de un motor de grafos nativo como Neo4j. Mediante índices compuestos y ordenamiento de IDs, Postgres resuelve estas consultas de forma directa (sin recorridos recursivos), con un costo operativo y de despliegue significativamente menor.
- **Arquitectura Híbrida de IA:** La IA no actúa como la base de datos (evitando alucinaciones o respuestas lentas en navegación UI), sino como un potenciador en dos fases: compilación de dataset en pipeline offline y generación de prosa culinaria en línea.

3. Arquitectura General del Sistema

El sistema utiliza un patrón de **Arquitectura Multicapa Desacoplada** (Frontend Client, Backend API, Relational Storage & External Services).

3.1 Flujo de Datos Principal

1. **Carga Inicial del Grafo:** El cliente React solicita `GET /api/v1/graph`. FastAPI consulta PostgreSQL y retorna los nodos y enlaces activos.
2. **Exploración y Filtrado:** El usuario selecciona un nodo (ej: *Tomate*). El frontend resalta vecinos y solicita `GET /api/v1/ingredients/{id}/pairings`.
3. **Explicación con IA (Online):** Al presionar “¿Por qué combinan?”, el frontend invoca `POST /api/v1/ai/explain-pairing`. FastAPI utiliza la interfaz `LLMProvider` para generar un párrafo descriptivo con tono gastronómico.
4. **Recetas Relacionadas:** Al solicitar recetas para una combinación (ej: *Tomate + Albahaca + Ajo*), FastAPI consulta la caché local. Si no existe en caché, llama a la API de Spoonacular y guarda el resultado.

4. Especificaciones del Modelo de Datos y Estrategia de Grafos

4.1 Esquema Relacional de PostgreSQL (DDL)

```
-- 1. Tabla de Categorías de Ingredientes
CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE, -- ej: Frutas, Verduras, Hierbas, Carnes, Lácteos
  color_code VARCHAR(7) NOT NULL    -- Hex color para el grafo (#FF5733)
);

-- 2. Tabla de Ingredientes
CREATE TABLE ingredients (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  description TEXT,
  category_id INT REFERENCES categories(id) ON DELETE SET NULL,
  flavor_profile JSONB DEFAULT '{}':jsonb, -- ej: {"sweet": 0.2, "umami": 0.8}
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Tabla de Afinidad de Maridaje (Grafo de Aristas)
CREATE TABLE flavor_pairings (
  id SERIAL PRIMARY KEY,
  ingredient_a_id INT NOT NULL REFERENCES ingredients(id) ON DELETE CASCADE,
```

```

ingredient_b_id INT NOT NULL REFERENCES ingredients(id) ON DELETE CASCADE,
affinity_score NUMERIC(3,2) NOT NULL CHECK (affinity_score BETWEEN 0.00 AND 1.00),
ai_rationale TEXT, -- Explicación prediseñada durante el pipeline offline

-- Restricción para garantizar que ingredient_a_id siempre sea menor que ingredient_b_id
CONSTRAINT chk_ordered_pair CHECK (ingredient_a_id < ingredient_b_id),
CONSTRAINT uq_ingredient_pair UNIQUE (ingredient_a_id, ingredient_b_id)
);

-- Índices optimizados para búsquedas bidireccionales inmediatas
CREATE INDEX idx_pairings_a ON flavor_pairings(ingredient_a_id);
CREATE INDEX idx_pairings_b ON flavor_pairings(ingredient_b_id);
CREATE INDEX idx_pairings_score ON flavor_pairings(affinity_score DESC);

-- 4. Usuarios y Favoritos
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_favorite_pairings (
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    pairing_id INT REFERENCES flavor_pairings(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, pairing_id)
);

-- 5. Caché e Indexación Permanente de Recetas en el Servidor
CREATE TABLE recipes (
    id SERIAL PRIMARY KEY,
    external_id INT UNIQUE, -- ID devuelto por Spoonacular (opcional)
    title VARCHAR(255) NOT NULL,
    image_url TEXT,
    source_url TEXT,
    ready_in_minutes INT,
    servings INT,
    raw_json JSONB NOT NULL, -- Datos completos de la receta (ingredientes, pasos, nutrición)
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE recipe_search_cache (
    id SERIAL PRIMARY KEY,
    cache_key VARCHAR(64) UNIQUE NOT NULL, -- Hash SHA-256 de los ingredient_ids ordenados

```

```

ingredient_ids INT[] NOT NULL,
recipe_ids INT[] NOT NULL, -- Arreglo de IDs de la tabla 'recipes'
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
last_accessed_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_recipe_search_hash ON recipe_search_cache(cache_key);

```

4.2 Lógica de Consulta Bidireccional

Dado que un par (ej: *Tomate, Albahaca*) es equivalente a (*Albahaca, Tomate*), la restricción `ingredient_a_id < ingredient_b_id` evita duplicados.

Para obtener todas las afinidades de un ingrediente `:ingredient_id`:

```

SELECT
  CASE
    WHEN p.ingredient_a_id = :ingredient_id THEN i2.id
    ELSE i1.id
  END AS neighbor_id,
  CASE
    WHEN p.ingredient_a_id = :ingredient_id THEN i2.name
    ELSE i1.name
  END AS neighbor_name,
  p.affinity_score,
  p.ai_rationale
FROM flavor_pairings p
JOIN ingredients i1 ON p.ingredient_a_id = i1.id
JOIN ingredients i2 ON p.ingredient_b_id = i2.id
WHERE p.ingredient_a_id = :ingredient_id OR p.ingredient_b_id = :ingredient_id
ORDER BY p.affinity_score DESC;

```

5. Pipeline Offline y Origen del Dataset de Sabores (Mes 1)

5.1 Origen y Fuentes Prácticas de la Información

El dataset no se construye por relevamiento manual, sino combinando tres fuentes complementarias:

1. **La IA como “Sintetizador y Destilador de Conocimiento” (Pipeline Offline - Método Principal):**
 - Los modelos de lenguaje modernos (OpenAI GPT-4o, Google Gemini) fueron entrenados con millones de textos científicos, recetas globales y literatura gastronómica de referencia (incluyendo *The Flavor Bible*, *The Flavor Thesaurus* y artículos científicos de maridaje molecular).

- En lugar de descargar o transcribir libros, nuestro script en Python (`scripts/seed_flavor_network.py`) consulta en lote al LLM mediante solicitudes estructuradas (JSON Mode con Pydantic). La IA actúa como un “chef experto”, evaluando cada par de ingredientes y generando la puntuación de afinidad (0.0 a 1.0) y la explicación culinaria.
2. **Datasets Abiertos Académicos en GitHub y Kaggle (Fuentes Públicas Abiertas):**
 - **FlavorDB / Flavornet:** Proyecto científico abierto de IIIT Delhi que mapea ~1,000 ingredientes a sus moléculas aromáticas volátiles (eugenol, linalool, etc.). Sus datasets son descargables públicamente en formato CSV/JSON.
 - **Nature Scientific Reports - Dataset de “Flavor Network”:** Dataset público del famoso estudio científico de Yong-Yeol Ahn (“*Flavor network and the principles of food pairing*”), disponible libremente en repositorios de GitHub.
 3. **Co-ocurrencia Estadística en Recetas (Spoonacular / RecipeDB):**
 - Mapeo estadístico automático: Si dos ingredientes (ej: *Tomate* y *Albahaca*) aparecen juntos frecuentemente en miles de recetas procesadas, se refuerza la puntuación de afinidad.

Nota metodológica sobre la normalización del score: las fuentes anteriores no son directamente comparables entre sí. FlavorDB/Flavornet expresan afinidad como cantidad de compuestos aromáticos volátiles compartidos (un número entero, no un score de 0 a 1), mientras que el LLM devuelve directamente un puntaje 0.0–1.0 y la co-ocurrencia en recetas es una frecuencia relativa. El pipeline define una fórmula explícita de normalización (por ejemplo, escalar la cantidad de compuestos compartidos contra el máximo observado en el dataset) para llevar todas las fuentes a la misma escala antes de promediarlas o combinarlas. Esta fórmula y su justificación deben documentarse como una decisión metodológica propia del proyecto.

5.2 Fases del Pipeline Offline (`scripts/seed_flavor_network.py`)

1. **Semilla de Ingredientes:** Listado inicial normalizado en JSON/CSV con ~250 ingredientes comunes clasificados por categorías (Frutas, Verduras, Carnes, Lácteos, Hierbas/Especias, Granos).
2. **Generación Automatizada de Pares:** El script genera pares lógicos de ingredientes y consulta al LLM en lotes para extraer puntajes de afinidad y explicaciones en español.
3. **Control de Calidad y Sanitización:**
 - Filtrado de pares con puntuación menor a 0.40 para evitar saturación visual en el grafo.
 - Validación de tipos con Pydantic.
 - Inserción ordenada en PostgreSQL (`ingredient_a_id < ingredient_b_id`).

6. ESPECIFICACIÓN DE LA API REST (FastAPI)

6.1 Autenticación (/api/v1/auth)

- POST /api/v1/auth/register: Registro de nuevo usuario.
- POST /api/v1/auth/login: Autenticación y retorno de Access Token JWT.
- GET /api/v1/auth/me: Perfil del usuario autenticado.

6.2 Red y Grafo (/api/v1/graph)

- GET /api/v1/graph: Retorna la estructura global de nodos y enlaces para react-force-graph.
 - **QueryParams:** min_affinity (default: 0.50), category_id (opcional).
- GET /api/v1/ingredients: Lista paginada con filtro de búsqueda de ingredientes.
- GET /api/v1/ingredients/{id}/pairings: Obtiene los ingredientes vecinos y sus afinidades.

6.3 Servicio de Evaluación Multi-Ingrediente e IA (/api/v1/pairings, /api/v1/ai)

Este servicio separa explícitamente dos responsabilidades: el cálculo de sinergia es **determinístico** (se resuelve enteramente contra los datos de `flavor_pairings`, sin invocar un LLM), mientras que la sugerencia de reemplazo es **generativa** (requiere una llamada a IA, ya que implica razonar sobre qué ingrediente alternativo mejoraría la combinación, algo que no se desprende directamente de la matriz de puntajes).

- POST /api/v1/pairings/evaluate: Recibe un arreglo de 2 o más `ingredient_ids`. Calcula, únicamente a partir de la base de datos, la matriz de afinidades cruzadas, el **Índice de Sinergia Global (0-100%)** y el o los ingredientes discordantes (*clashing elements*, definidos como el ingrediente con menor afinidad promedio respecto al resto del grupo).
- POST /api/v1/ai/suggest-replacement: Recibe el resultado de `evaluate` cuando se detecta un ingrediente discordante. Invoca al LLM con ese contexto (afinidades ya calculadas) para sugerir un reemplazo con sentido gastronómico y redactar la justificación. Se llama solo bajo demanda del usuario (botón explícito), no automáticamente en cada evaluación.
- POST /api/v1/ai/explain-pairing: Recibe `[ingredient_id_1, ingredient_id_2, ...]`. Invoca la interfaz de LLM y devuelve la explicación organoléptica en tiempo real.

```
// POST /api/v1/pairings/evaluate Body:
{
  "ingredient_ids": [12, 45, 88] // ej: Tomate, Albahaca, Chocolate
```

```

}

// Response 200 OK (cálculo determinístico, sin IA):
{
  "overall_synergy_score": 58.5,
  "synergy_label": "Maridaje Moderado / Arriesgado",
  "pairwise_matrix": [
    {"pair": ["Tomate", "Albahaca"], "affinity": 0.96, "status": "excellent"},
    {"pair": ["Tomate", "Chocolate"], "affinity": 0.35, "status": "clash"},
    {"pair": ["Albahaca", "Chocolate"], "affinity": 0.44, "status": "weak"}
  ],
  "clashing_ingredients": ["Chocolate"]
}

// POST /api/v1/ai/suggest-replacement Body (solo si el usuario lo solicita):
{
  "ingredient_ids": [12, 45, 88],
  "clashing_ingredient": "Chocolate"
}

// Response 200 OK (generado por LLM):
{
  "recommendation": "El Chocolate presenta baja compatibilidad con el Tomate. Se sugiere ree"
}

```

6.4 Servicio de Recetas y Estrategia de Caché Permanente (/api/v1/recipes)

- GET /api/v1/recipes/search?ingredient_ids=12,45,88: Recibe una lista de ingredientes y busca recetas coincidentes.

¿La información es temporal (usuario) o permanente (servidor)? La información **QUEDA GUARDADA DE FORMA PERMANENTE EN EL SERVIDOR** (Base de Datos PostgreSQL). No es temporal del navegador del usuario.

Razones Técnicas de esta Decisión:

1. **Ahorro de Cuota de API (Límite Spoonacular):** La cuota gratuita de Spoonacular ofrece solo 150 puntos/día. Guardar las recetas en el servidor evita agotar la cuota con búsquedas repetidas.
2. **Reducción de Latencia:** Una búsqueda a Spoonacular tarda entre 800ms y 2000ms. Consultar recetas ya cacheadas en PostgreSQL es sensiblemente más rápido, al tratarse de una lectura local sin llamada HTTP externa.
3. **Construcción Progresiva del Dataset:** Con el uso diario de los usuarios, el servidor va construyendo automáticamente su propio repositorio

enriquecido de recetas.

Idioma de las Recetas y Traducción Automática al Español La base de datos original de **Spoonacular** está principalmente en inglés. Para ofrecer una experiencia 100% nativa en español:

1. **Pipeline de Traducción al Cachear (Translation-on-Cache):**
 - Cuando FastAPI recupera una receta nueva de Spoonacular (en inglés), **antes de guardarla en PostgreSQL**, el backend realiza un pase automático rápido por el servicio de LLM (Gemini Flash / GPT-4o-mini) con el prompt: “*Traduce al español neutro el título, ingredientes e instrucciones de preparación manteniendo la estructura JSON*”.
2. **Cero Latencia Adicional para el Usuario:**
 - La traducción ocurre **una sola vez por receta** (al momento de ser descubierta e ingresada a la tabla **recipes**).
 - Todas las consultas posteriores leen el texto traducido directamente desde PostgreSQL en $< 10\text{ms}$.

Flujo de Obtención, Traducción e Indexación (FastAPI Backend):

7. Diseño Frontend y Experiencia Visual

El frontend se estructurará con **React (Vite)** y **Tailwind CSS**.

7.1 Visualización de Compatibilidad Multi-Ingrediente

1. **Modo Grafo Enfocado (Sub-graph Spotlight):**
 - Al seleccionar 2 o más ingredientes, el grafo atenúa los nodos no relacionados (*dimming*) y resalta el subgrafo formado por los ingredientes seleccionados.
 - **Codificación de Colores de Aristas (Enlaces):**
 - **Verde esmeralda** ($> 75\%$): Maridaje armónico/excelente.
 - **Amarillo / Naranja** ($45\% - 74\%$): Afinidad neutra o secundaria.
 - **Rojo Punteado** ($< 45\%$): Choque de sabor / baja incompatibilidad (*clash*).
2. **Medidor Visual de Sinergia (Synergy Gauge):**
 - Un indicador visual (barras de porcentaje o medidor semicircular) en el panel lateral que indica el score de maridaje del plato/receta completa.
3. **Matriz Interactiva $N \times N$:**
 - Una micro-tabla o mapa de calor (*heatmap*) en el panel lateral que permite tocar cualquier par para ver su justificación organoléptica individual.
4. **Detector del Elemento Discordante (*Clashing Ingredient Alert*):**

- Si el usuario ingresa 3 ingredientes que combinan bien (ej: *Tomate, Queso, Albahaca*) y 1 que desentona (ej: *Café*), el sistema resalta visualmente el nodo discordante en rojo con una alerta: “*El ingrediente Café disminuye la sinergia general en un 35%*”.

5. Interacciones:

- Click en nodo: Aplica zoom suave y abre el Drawer con detalles del ingrediente.
- Hover en arista: Muestra tooltip con el % de afinidad.
- Selector de Categorías: Filtra y destaca nodos por color en tiempo real.

8. Plan Global de Implementación y Cronograma (4 Meses)

Detalle de Fases:

Mes 1: Data Science y Cimientos

- Creación de base de datos en PostgreSQL.
- Desarrollo del script `seed_flavor_network.py` con integración de API de IA.
- Carga de dataset validado (~250 ingredientes, ~1,500 relaciones).

Mes 2: Backend Core y Servicios

- Implementación de endpoints REST en FastAPI.
- Seguridad con JWT (login/registro).
- Cliente de Spoonacular con almacenamiento en caché Postgres para proteger cuota de uso.

Mes 3: Frontend y Grafo Interactivo

- Integración de `react-force-graph-2d` en React.
- Implementación de filtros dinámicos por categoría y rango de afinidad.
- Paneles laterales (drawers) de información y favoritos del usuario.

Mes 4: Integración Online de IA, Testing y Defensa

- Botón de explicación en vivo con LLMs (Gemini/OpenAI).
 - Pruebas de integración, optimización de velocidad de carga de la base de datos y UI.
 - **Stretch Goal (Opcional):** Fine-tuning liviano de Llama 3.2 3B en Google Colab con exportación a GGUF para demostración local en Ollama.
-

9. Matriz de Riesgos y Mitigación

Riesgo Identificado	Impacto	Mitigación Planificada
Agotamiento de cuota en API de Recetas	Medio	Implementación obligatoria de tabla caché en PostgreSQL; límite de solicitudes en frontend.
Saturación visual en el grafo por demasiados nodos	Alto	Filtro estricto en frontend con <code>min_affinity</code> por defecto en 0.50 y paginación de nodos vecinos.
Alucinaciones o latencia en respuestas de IA	Medio	Prompting estructurado con Pydantic/JSON Mode; fallback a la justificación pre-calculada en la DB si la llamada API falla o sobrepasa el timeout (3s).
Demoras en el objetivo opcional (Modelo Local)	Bajo	El modelo local es un <i>Stretch Goal</i> . Si no se completa a tiempo, el proyecto principal con API Cloud se mantiene 100% funcional.

10. Análisis de Viabilidad Económica y Escalabilidad

El diseño arquitectónico del proyecto garantiza una **alta eficiencia de costos y sostenibilidad financiera**, permitiendo operar a costo **\$0.00 USD** durante toda la fase de desarrollo y defensa, manteniendo costos marginales mínimos ante un crecimiento de usuarios a escala.

10.1 Estrategia de Optimización de Costos de API

- API de Recetas (Spoonacular) — Amortización por Caché Local:**
 - La cuota gratuita ofrece 150 puntos/día.
 - Gracias al **Caché Progresivo en PostgreSQL (`recipe_search_cache`)**, la dependencia de la API disminuye asintóticamente con el uso: más del 90% de las búsquedas frecuentes de usuarios leen directamente de la base de datos local (< 10ms) sin consumir cuota externa.

2. **APIs de Inteligencia Artificial (LLM) — Modelos de Alta Eficiencia:**

- Se utilizan modelos de última generación ultralivianos (Google Gemini 1.5 Flash / OpenAI GPT-4o-mini).
- **Generación en tiempo real:** \$0.075 USD por millón de tokens en Gemini Flash (~\$0.00005 USD por explicación de chef).
- **Pipeline Offline (Mes 1):** Compilación inicial del grafo de 1,500 afinidades por **~\$0.30 USD por única vez.**

10.2 Cuadro Comparativo de Proyección de Costos por Nivel de Escala

Nivel de Escala	Usuarios Activos / Mes	API Recetas (Con Caché DB)	API IA (Gemini Flash / GPT-4o-mini)	Hosting & Post-greSQL	Costo Total Estimado
MVP / Defensa Tesis	1 – 100	\$0.00 (Free Tier)	\$0.00 (Free Tier)	\$0.00 (Render / Supabase Free)	\$0.00 USD / mes
Producción Inicial	1,000	\$0.00 (Caché DB absorbe 95%)	~\$0.15 USD	\$0 – \$5.00 USD	~ \$0.15 – \$5.00 USD / mes
Escala Media	25,000	~\$29.00 USD (Spoonacular Builder)	~\$2.50 USD	~\$10.00 USD (DB 5GB)	~ \$41.50 USD / mes

10.3 Argumentación de Viabilidad para la Defensa Academic

Este análisis demuestra criterio de ingeniería de software enfocado en la **economía de recursos y optimización operativa**, probando que el sistema no solo es funcional y estéticamente atractivo, sino también **financieramente viable y preparado para producción real.**